

Basic (your first real API call) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real caller reuses the client and guards against a reply with no text). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The three pieces, and what each returns](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The three pieces, and what each returns

Basic Version

You need three things: a client (your connection to OpenAI), a list of messages (what you're saying to it), and one line to print the reply.

The client reads your API key automatically from the `OPENAI_API_KEY` environment variable, if `.env` is already loaded — you don't have to type the key into your code anywhere.

The reply text is buried a few levels deep in the response object:

`response.choices[0].message.content` . There's always at least one item in `choices` , so `[0]` is safe here.

Pick two very different system prompts to compare — like "You are a formal, professional assistant" versus "You are a sarcastic pirate" — so the change in the answer is obvious.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Three pieces, matching the Core Concepts above: `client = OpenAI()` creates the client (it reads `OPENAI_API_KEY` from the environment on its own, so call `load_dotenv()` from Doc01's config pattern first if you haven't already). `client.chat.completions.create(model=..., messages=[...])` sends the request. The `messages` list needs at least one `{"role": "system", ...}` entry and one `{"role": "user", ...}` entry — the system entry sets behavior, the user entry is the actual question.

`response.choices` is a list because the API can technically return more than one candidate answer (controlled by an `n` parameter you're not using here) — with the default settings, there's exactly one, at index `0`. `response.choices[0].message.content` is a plain string: the model's reply text.

For the system-prompt comparison, keep the user message identical across both runs and change only the system message — that isolates the variable you're actually testing.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

This exercise only asks for one call, so it's easy to write `client = OpenAI()` inside a function and never think about it again. But ask: **what happens the moment this code gets called from more than one place** — a second exercise, a test file, `main.py` ? Building a brand-new client object on every single call is wasteful (it re-reads environment variables and sets up connection settings each time), and it means every caller has to remember to do it correctly.

There's also a real gap in `response.choices[0].message.content` : it can actually be `None` , not just a string — this happens when the model's turn ends for a reason other than "wrote normal text," like being cut off, or (once you reach tool-calling documents) making a tool call instead of replying in words. A script that always assumes `content` is a string will crash confusingly, deep inside whatever code tries to use it, the one time this happens.

Two extra pieces answer both problems:

- **A module-level cached client** — the exact same `_config / global` caching pattern from Doc01's Build Task, applied to `OpenAI()` instead of `Config` . Build the client once, the first time it's needed, and hand back that same object every time after.
- **A `finish_reason` check** — `response.choices[0].finish_reason` tells you *why* the model stopped (`"stop"` is the normal case). Checking it, and raising a clear error when `content` is `None` , turns a confusing `TypeError` somewhere downstream into an obvious, immediate one naming exactly what happened.

```
# chat_api_basics_practice.py
_client = None

def get_client():
    global _client
    if _client is None:
        load_dotenv()
        _client = OpenAI()
    return _client
```

Sketch the `finish_reason` check yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the three pieces and where the reply text lives, for the tidy one-shot case. Intermediate explains exactly what each piece is doing and why `choices` is a list at all. Advanced asks what changes once this code is called from more than one place, and once the model's reply isn't guaranteed to be plain text — a cached client

(so every caller shares one connection instead of building their own) and a `finish_reason` check (so a missing reply fails loudly, immediately, instead of crashing confusingly somewhere else).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
import the OpenAI client
load the .env file

make the client

messages = [
    system message: "You are a formal, professional assistant."
    user message: "Tell me about your day."
]

send the messages to the model
print the reply text

change the system message to something very different
send again with the same user message
print the new reply text
```

Here's almost the whole thing for the first call — just try running it and reading it line by line:

```
# chat_api_basics_practice.py
from openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a formal, professional assistant."},
        {"role": "user", "content": "Tell me about your day."},
    ],
)
print(response.choices[0].message.content)
```

Expected output if you run just this: a formal, professional-sounding reply about "having a day," since language models play along with the premise. What's missing: loading `.env` first, and the

second call with a different system prompt.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

function ask(system_prompt, user_prompt) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    return response.choices[0].message.content

print(ask("You are a formal, professional assistant.", "Tell me about your day."))
print(ask("You are a sarcastic pirate.", "Tell me about your day."))
```

Wrapping the call in a small `ask()` function is optional for this exercise but makes comparing two system prompts much less repetitive. Write it this way, run it, and confirm the two replies clearly differ in tone before moving on.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```

module-level: _client = None

function get_client():
    if _client is already set: return it
    load_dotenv()
    build the client, store it in _client
    return it

function ask(system_prompt, user_prompt) -> str:
    client = get_client()
    response = client.chat.completions.create(...)
    choice = response.choices[0]
    if choice.message.content is None:
        raise a clear error naming choice.finish_reason
    return choice.message.content

```

Turning that into real code — fill in the missing piece yourself:

```

# chat_api_basics_practice.py
from dotenv import load_dotenv
from openai import OpenAI

_client: "OpenAI | None" = None

def get_client() -> OpenAI:
    global _client
    if _client is None:
        load_dotenv()
        _client = OpenAI()
    return _client

def ask(system_prompt: str, user_prompt: str) -> str:
    client = get_client()
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_prompt},

```

```

        {"role": "user", "content": user_prompt},
    ],
)
choice = response.choices[0]
# your turn: if choice.message.content is None, raise a clear
# RuntimeError naming choice.finish_reason instead of returning None
...
return choice.message.content

```

Expected output if you call `get_client()` twice in a row: the exact same object both times — prove it with `get_client() is get_client()`, which should print `True`.

Fill in the `finish_reason` check yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic's near-complete code makes one call and prints the reply, with nothing shared between calls. Intermediate wraps the same call in a reusable `ask()` function, so comparing prompts is a one-liner instead of a copy-pasted block. Advanced adds a cached client underneath `ask()` (so a second, third, or hundredth call reuses the same connection instead of rebuilding it) and a guard on the reply itself — the same "don't trust the happy path silently" idea from Hint 1's Advanced question, now written as code.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)